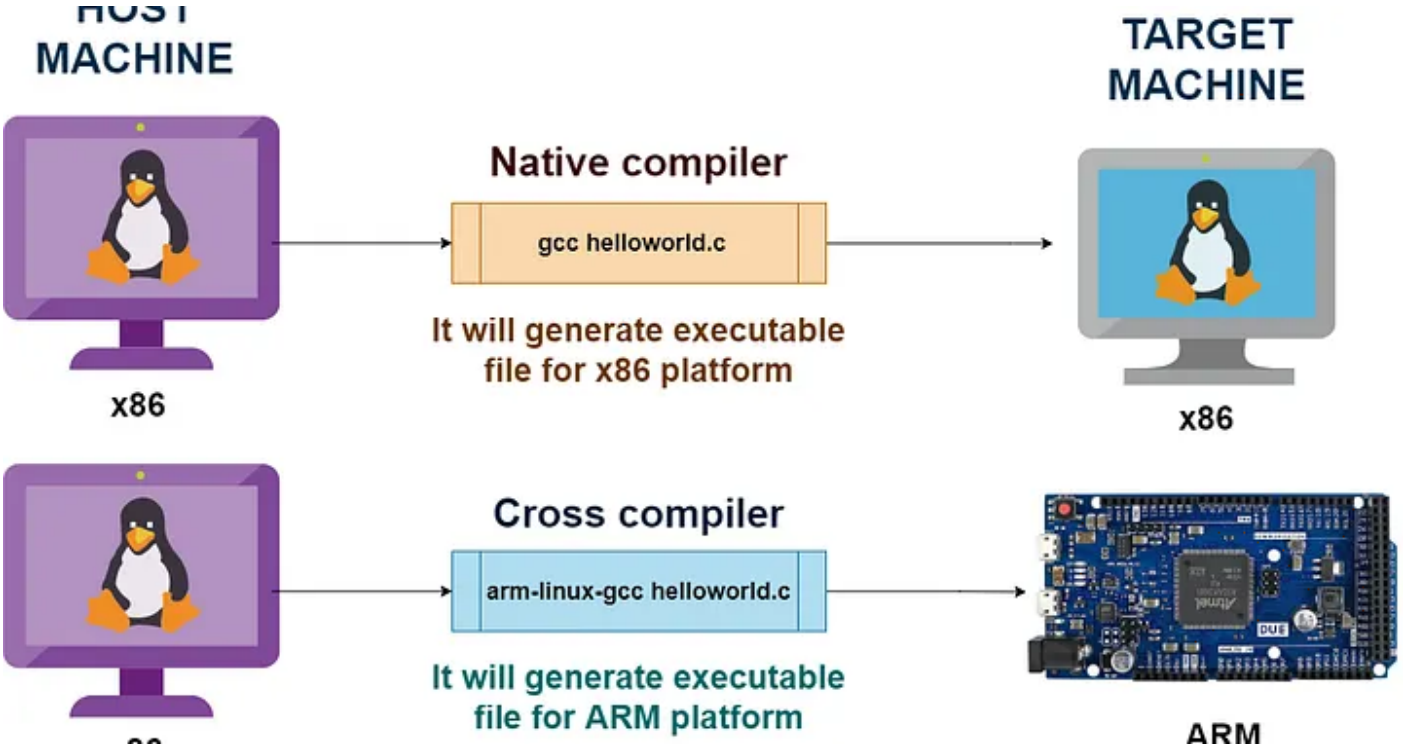
[Huyen Nguyen] Toolchain & Cross-Compilation

1. Cross-compiling toolchain



Native Compiler

- Compiles source code on a machine such that the resulting executable can only run on that exact same machine architecture.

Cross compiler

- Runs on a development machine (Host—typically an Intel/AMD x86 chip), but generates machine code specifically for a different processor (Target—such as an ARM chip on a Raspberry Pi).

Cross-compiler system

- BUILD: The computer used to build the toolchain (e.g., an x86_64 machine).
- HOST: The computer that will run the toolchain to compile the software source code.

- TARGET: The actual board that will run the compiled program file (e.g., a Raspberry Pi board with an ARM chip).

Core components of a cross-toolchain

Binutils (Binary Utilities): A collection of tools for manipulating the target's machine code files

- as (Assembler): Translates Assembly code into object machine code (.o).
- ld (Linker): Links object files (.o) and libraries together into a complete executable file.
- ar & ranlib: Packages object files (.o) into static library files (.a).
- objdump / readelf: Inspects the internal structure of ELF files (reads headers, views assembly code, checks dependencies).
- strip: Strips away all unnecessary variable and function names (debug symbols) to reduce the binary file size before flashing it to the board.

C/C++ Library

- Acts as an intermediate layer between applications and the kernel.
- Provides standard functions and translates them into system calls down to the kernel.
- Examples: glibc, musl, uClibc.

Kernel Headers

- A set of .h header files extracted from the Linux Kernel source code.
- Provides data structure definitions, constants, and function prototypes that applications cannot easily define themselves or are too complex to rewrite.

GCC Compiler: The core compiler that translates source code into machine code.

GDB Debugger

- Consists of cross-gdb running on the host machine and gdbserver running on the target board.
- Enables developers to set breakpoints, monitor variables, and debug programs running on the embedded board directly from their personal computers.

Comparing Pre-built Toolchain vs. Custom-built Toolchain

Pre-built Toolchain

- Availability: Ready to use out of the box.
- Hardware Optimization: Generic; usually configured with safe defaults for a wide range of devices -> fails to fully leverage target-specific chip features.
- Compatibility Control: Difficult to manage; easily leads to glibc version mismatch issues between the host/build machine and the target board.
- Stability: High stability, thoroughly tested by major vendors and communities.

Custom-built Toolchain

- Setup Effort: Requires detailed configuration and time-consuming compilation.
- Hardware Optimization: Maximally optimized for the specific architecture and hardware capabilities.
- Compatibility & Accuracy: Highly precise; allows exact selection of libc versions and Kernel headers matched perfectly to the target board.
- Stability: Depends heavily on the developer's configuration and expertise.

2. glibc / uClibc / musl

glibc (GNU C Library)

- Developed by the GNU Project; serves as the default standard C library for Linux distributions like Ubuntu, CentOS, etc.
- Supports all standards (POSIX, C99, C11, C23) along with numerous GNU-exclusive extensions.
- Supports internationalization (i18n / multi-language support).
- Large file size.
- 100% standard compatibility.
- Heavyweight, high RAM consumption, slower startup times.
- Best used for: Hardware devices with rich resources (e.g., Raspberry Pi, SBCs).

uClibc

- Designed for ultra-resource-constrained embedded systems (such as microcontrollers running uClinux, legacy ARM chips, and MIPS chips).
- Provides essential libc functionality while heavily optimized for size (10 to 20 times smaller than glibc).
- Strips away many GNU extensions to conserve RAM. Due to extensive trimming, compiling complex modern software (such as Python 3, Node.js, or modern C++) frequently fails because those required functions are absent.
- Supports non-MMU chips (processors without a Memory Management Unit, such as ARM Cortex-M/R cores).

musl

- Designed with the goal of: Being as lightweight as uClibc while remaining strictly standard-compliant and more modern than glibc.
- Compact size: Typically 600 KB to 900 KB.
- Strict compliance with international POSIX standards.
- Best used for: Next-generation IoT devices where the entire OS footprint needs to be kept around a few tens of megabytes.

3.

Kernel Headers in a toolchain must not be newer than the Kernel running on the target board: Newer kernels are always backward-compatible with code built against older kernel headers, but an older running kernel cannot recognize or support features introduced in newer kernel headers.

Why is a sysroot required in cross-compilation?

- All header files in /usr/include or libraries in /lib on the host machine are intended for Intel/AMD x86_64 chips.
- If the cross-compiler mistakenly pulls these files, the compiled program for the ARM board will become corrupted or target the wrong architecture.

=> Therefore, the toolchain needs to establish a dedicated directory called a sysroot, which contains the system files for the ARM chip to ensure correct compilation.

Internal structure of sysroot

```
sysroot/
├── lib/                <-- Contains the minimal C library and Dynamic Linker
│   ├── ld-musl-armhf.so.1
│   └── libc.so
│
└── usr/                <-- Contains Header files and Extended libraries
├── include/       (Houses stdio.h, stdlib.h, unistd.h, linux/..., asm/...)
└── lib/           (Houses startup object files like crt1.o, crti.o, and libc.a)
```

When is sysroot used?

- During compilation and linking.
- When running/emulating tests on QEMU.

Build a custom cross-toolchain for target

- Use the Crosstool-NG tool to compile from source, creating an independent Cross-Toolchain optimized specifically for the target board's hardware.
- Ensure a clean compilation environment, independent of the Host operating system's libraries (Ubuntu x86_64).
- Target Board: ARM Versatile Express Cortex-A9 (QEMU vexpress-a9)

Compile, run test program & verify target architecture

Test source code

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    printf("Hello world!\n");
    return EXIT_SUCCESS;
}
```

Output

```
hello: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter
/lib/ld-musl-armhf.so.1, not stripped
```

- Standard Linux executable file format, 32-bit, Little Endian (byte order of Cortex-A9).

Run tests using QEMU User-Mode Emulation

```
qemu-arm -L ~/x-tools/arm-training-linux-musleabihf/arm-training-linux-musleabihf/sysroot
hello
```

Output

```
Hello world!
```

- qemu-arm acts as a CPU interpreter, reading each ARM machine code instruction and performing dynamic translation into x86_64 code directly within the Host machine's RAM, then forwarding system calls to the Host's Linux Kernel to print the string to the screen

---

Revision #2
Created 2026-10-21 01:46:37 UTC by Huyen Nguyen
Updated 2026-10-22 02:11:26 UTC by Huyen Nguyen